



SDLC using Agile Methodology

Agile Methodology

- **People collaborating together to build high quality products that meet customer needs at a sustainable pace**
- **The tasks are divided to time boxes (small time frames) to deliver specific features for a release**
- **Each delivery is incremental in terms of features; the final delivery holds all the features required by the customer**
- **Iterative approach is taken and working software is delivered after each iteration**



Agile Process Principles/Agile Manifesto

1. Highest priority is to **satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing** requirements, even late in development.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must **work together daily** throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.



Agile Process Principles

- 
7. **Working software** is the primary measure of progress
 8. Agile processes promote **sustainable**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely
 9. **Continuous attention** to technical excellence and good design enhances agility
 10. **Simplicity** - the art of maximizing the amount of work done--is essential
 11. The best architectures, requirements, and designs emerge from **self-organizing teams**
 12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly (**Frequent Review**).

Agile using SCRUM



Scrum is one of the Agile development model. In the scrum model, the entire project work is divided into small work parts that can incrementally be developed and delivered over time boxes. These time boxes are called sprint.

- At the end of each sprint, the stake holder and the team members **meet to assess the developed s/w increment**. The stake holder may suggest any changes and improvements that they might feel necessary.
- In scrum, the s/w gets developed over a series of sprints. In each sprint, increments to the s/w are **deployed to the client site** and also client **feed back** is obtained.

Key Roles and Responsibilities

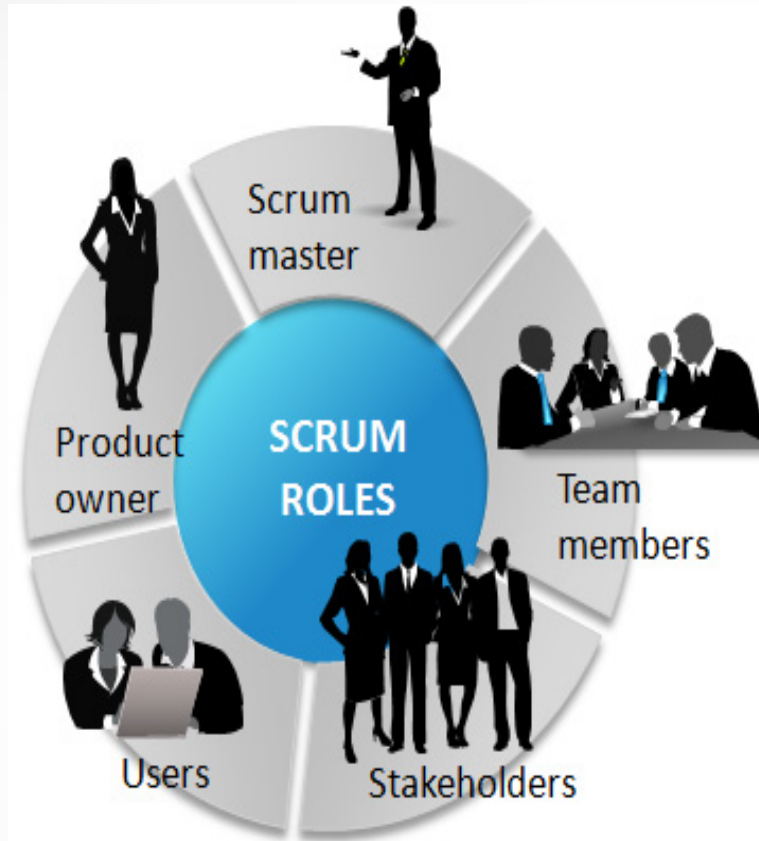
Scrum Master: The scrum can set up the master team, arrange the meeting and remove obstacles for the process

Product owner: The product owner represents the customer's perspective and guides the team toward building the right software. The product owner is responsible for **communicating the customer's view** of the software to the development team. In every sprint, the product owner in consultation with the team members defines the features of the software to be developed in the next sprint, decides on the release dates, and also may reprioritize the required features that are yet to be developed if necessary.

Team member: A scrum team usually consists of cross-functional team members with expertise in areas such as quality assurance, programming, user interface design, and testing. The team is self-organising in the sense that the team members distribute the responsibilities among themselves. This is in contrast to a conventional team where a designated team leader decides who will do what.



SCRUM Roles



Product Owner

- Represents the Business side of the Product / Software Development

Scrum Master

- Represents the People side of the Product/Software Development

Development Team

- Represents the Technology side of the Product/Software Development

Product Owner

Represents the business side of the Product/Software Development



The Product Owner is responsible for maximizing the value of the product and the work of the Development Team

The Product Owner is the sole person responsible for managing the Product Backlog

Creating Product Vision

Stakeholder Management

Scope Management

Release Management

Scrum Master



Vision: Understand the product vision and lead/coach the team to work towards it

Product Backlog Refinement: Help PO in defining and maintaining the product backlog

Sprint Planning: Facilitate the Sprint Planning meeting & help team to create the Sprint Backlog

Development: Coach team to self-organize and help them to resolve the Impediments

DSM: Facilitate and help the team to inspect the progress towards the sprint goal

Sprint Review: Facilitate and ensure that the focus is on integrated product increments

Sprint Retrospective: Facilitate and help team to inspect and adapt the process

Development Team

The Development Team is responsible for delivering a potentially shippable increment of working software

Develop high quality solution to fulfil the product/software vision

The Development Team is Self-organized

The Development Team is Cross functional

The Development Team is Empowered & autonomous

Collectively responsible for delivery

Size will be between 6 to 7 (recommended)



Scrum ceremonies



Scrum ceremonies are structured meetings that provide a framework for teams to work within the Scrum methodology. These ceremonies ensure consistent communication, planning, and review throughout the development process. Here are the key Scrum ceremonies:

1. Sprint Planning

Purpose: Plan the work to be completed in the upcoming sprint.

When: At the beginning of each sprint.

Participants: Product Owner, Scrum Master, Development Team.

Activities:

The Product Owner presents the prioritized backlog items.

The team discusses and estimates the effort required for each item.

The team commits to the tasks they can realistically complete in the sprint.



2. Daily Scrum (Daily Stand-up)

Purpose: Synchronize the team's activities and plan for the next 24 hours.

When: Daily, typically at the same time and place.

Participants: Development Team, Scrum Master (optional), Product Owner (optional).

Activities:

Each team member answers three questions:

What did I do yesterday?

What will I do today?

Are there any impediments or blockers?

3. Sprint Review

Purpose: Demonstrate the work completed during the sprint and gather feedback.

When: At the end of each sprint.

Participants: Product Owner, Scrum Master, Development Team, Stakeholders.

Activities:

The team demonstrates the potentially shippable product increment.

Stakeholders provide feedback and discuss any changes or new requirements.

The Product Owner updates the backlog based on the feedback received.

Scrum Practices

Scrum relies on a set of core practices to help teams deliver value incrementally while promoting collaboration and continuous improvement.

Core Scrum Practices

- Empirical Process Control – Based on transparency, inspection, and adaptation
- Timeboxing – Fixed time duration for events and development cycles
- Iterative Development – Work delivered through repeated cycles (Sprints)
- Incremental Delivery – Frequent delivery of functional increments
- Prioritized Backlog – Work arranged by value and urgency
- Cross-functional Self-organizing Teams – Teams decide how to accomplish work
- Daily Scrum – Short daily meeting for alignment and issue resolution
- Sprint Reviews & Retrospectives – Demonstration and process improvement
- Definition of Done (DoD) – Shared understanding of completion
- Product Increment – Usable and potentially shippable product after every sprint



Applying Scrum

Scrum can be applied in project environments that require flexibility and rapid response to changes.

Steps to Apply Scrum Effectively

1. Define Product Vision and Roadmap
2. Form the Scrum Team (Product Owner, Scrum Master, Development Team)
3. Create and Prioritize Product Backlog
4. Sprint Planning – Select items for a sprint
5. Execute Sprint – Develop product increment
6. Daily Scrum – Track progress and remove blockers
7. Sprint Review – Present completed work to stakeholders
8. Sprint Retrospective – Identify improvements for next sprint
9. Continuous Integration and Testing
10. Repeat cycle until completion



Key Success Factors in Applying Scrum



- Leadership support and team empowerment
- Active customer involvement
- Strong collaboration and communication
- Continuous improvement mindset
- Use of Agile tools (Jira, Trello, Azure DevOps)

Need for Scrum

- Scrum has become essential in modern software development as organizations demand faster, flexible, and customer-centric delivery models.



Reasons Why Scrum is Needed

- Rapidly changing business requirements
- Need for faster product delivery and reduced time-to-market
- Encourages teamwork and ownership
- Enhances product quality through regular testing and reviews
- Early detection of risks and problems
- Increased customer satisfaction due to regular visibility
- Better predictability and planning accuracy
- Adaptable for complex, uncertain, and innovative projects

When Scrum Should be Used

- Projects with frequently changing requirements
- Products requiring continuous upgrades and improvements
- Complex and uncertain development environments
- Cross-functional collaborative teams

How Scrum Works – Step-by-Step Flow



1. Product Vision – Defined by the Product Owner
2. Product Backlog Creation – High-level requirements prioritized by value
3. Sprint Planning – Team selects items (User Stories) to work on
4. Sprint Execution – Development, testing, and integration activities
5. Daily Scrum – 15-minute stand-up meeting for progress and issues
6. Sprint Review – Demonstration of increment to stakeholders
7. Sprint Retrospective – Review team performance and improvements
8. Product Increment Release – Working version delivered after each sprint

Advanced Practices and Scaling Methods



- Scrum of Scrums – Coordination across multiple teams
- Scaled Agile Framework (SAFe) – Enterprise-level scaling
- Large Scale Scrum (LeSS) – Simplified scaling for multiple teams working on one product
- Nexus Framework – Integration management across teams
- Disciplined Agile Delivery (DAD) – Hybrid approach combining Agile methods
- Agile Release Trains (ART) – Teams working in alignment for product delivery

Advanced Concepts

- Release Planning & Roadmaps
- DevOps integration & CI/CD pipelines
- Metrics: Velocity, Burndown Chart, Burnup Chart
- Agile Maturity and Continuous Improvement Culture

Scrum and the Organization

Scrum positively transforms the organization by improving agility, communication, and delivery efficiency.

Impact of Scrum on Organizations

- Encourages transparency and collaboration
- Aligns teams with customer and business goals
- Improves productivity and product quality
- Faster delivery and better risk management
- Enables continuous learning and adaptation

Organizational Requirements for Successful Scrum

- Supportive leadership and culture
- Empowered cross-functional teams
- Open communication and stakeholder involvement
- Investment in training, tools, and coaching



Scrum Values



Scrum is built on five core values that guide team behavior and decision-making.

The Five Scrum Values

1. Commitment – Dedication to goals and teamwork
2. Courage – Speak openly, take risks, and face challenges
3. Focus – Concentrate on sprint goals and priorities
4. Openness – Transparent sharing of progress, problems, and ideas
5. Respect – Value and support each other as capable professionals

Importance of Scrum Values

- Strengthens teamwork and trust
- Reduces conflicts and improves collaboration
- Enhances productivity and motivation
- Builds a positive and high-performing workplace environment

Agile v/s Waterfall



Agile Development

Allows interim change in customer development

Iterative and Incremental

Produces a workable software with every milestone

Requires less cumbersome documentation

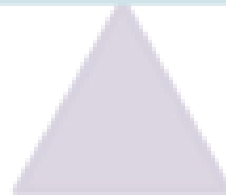
Waterfall Model

Adheres to the plan developed at the beginning of project

Phased and Sequential

Only a final product is delivered at the end

Emphasis on clear and elaborate documentation



Extreme Programming Model

The name of this model reflects the fact that it promotes taking best practices that have previously worked well in projects to extremes. This model is founded on a simple philosophy: "If something is known to be beneficial, why not put it to constant use?" Based on this principle, it proposes several key practices that must be taken to the extreme.



Good practices that need to be carried on to the extreme



Code review: Code review is beneficial because it allows for the most efficient detection and correction of problems. XP recommends pair programming as a method for achieving continuous review. Pair programming involves two programmers working together to code. In each pair, the two programmers take turns writing code. While one writes code, the other reviews it.

Testing: Testing code helps to eliminate problems and improves reliability. XP recommends Test-Driven Development (TDD) for continuously writing and executing test cases. The TDD approach involves writing test cases for a feature before writing any code for it. That is, before writing code to implement a feature, test cases are created based on user stories. Only when these tests pass is a piece of code considered complete.



Incremental development: Incremental development is beneficial since it helps to get user input, and the number of features produced is a dependable sign of progress. It advises that the team create fresh increments every few days.

Simplicity: Simple code produces high-quality results. It also simplifies testing and debugging the code. As a result, one should aim to write the simplest code possible while ensuring that the essential functionality being implemented works. Efficiency, reliability, maintainability, and other factors should be ignored when writing the simplest code possible. Once the basic code is functional, reworking might incorporate additional desirable features.

The 12 Key Practices of XP

- 1.Planning Game – Quick planning between customer and developers to decide scope and priority.
- 2.Small Releases – Frequent releases to production to gather real customer feedback.
- 3.System Metaphor – A simple shared vision of how the system works.
- 4.Simple Design – Design only what is required for current needs; avoid complexity.
- 5.Testing (Test-Driven Development) – Write tests before writing code to ensure correctness.
- 6.Refactoring – Continuous improvement of code structure without changing behavior.
- 7.Pair Programming – Two developers work together at one workstation to improve code quality.
- 8.Collective Code Ownership – Any developer can change any part of the code at any time.
- 9.Continuous Integration – Frequently integrate and build code to detect issues early.
- 10.40-Hour Work Week (Sustainable Pace) – Maintain a balanced workload to

Applications:

1. Projects involving new technology or research project
 - Because the requirements change rapidly and unforeseen technical problems are often needed to be resolved.
2. Small Projects



Agile model versus iterative waterfall model



The waterfall model is a structured approach that progresses through defined stages: requirements, analysis, design, coding, and testing, measuring progress through completed artifacts. In contrast, the agile model focuses on delivering functional software frequently, assessing progress by the features developed. Despite their differences, agile teams often adopt a miniature waterfall cycle within each iteration. Thus, while distinct, the two methodologies can complement each other.

In contrast to the waterfall model, which yields only documents if a project is canceled midway, the agile model can provide customers with functional code that may already be operational.

Agile model versus RAD model

The important differences between the agile and the RAD models are the following:

- While the agile model focuses on systematically developing incremental features, the RAD model prioritizes creating quick prototypes that are refined into production-quality code.
- Agile projects incrementally develop and deliver features, while the RAD approach emphasizes initially creating all application features in a rough form, followed by successive improvements over time.
- Agile teams showcase only completed work to customers, whereas RAD teams present simplified screen mock-ups and prototypes that often omit complex computations.





..End..